TECHNICAL OVERVIEW · CHARTERED INVESTMENT CODING CHALLENGE

Custody Ops Console: how it is built and why

A back-office web app for Vellum Fund Services' operations team: clients, transaction approval, monthly fee invoicing and custodian file import. This document explains the problem as understood, the architecture, the data model and the mechanisms behind the hard requirements.

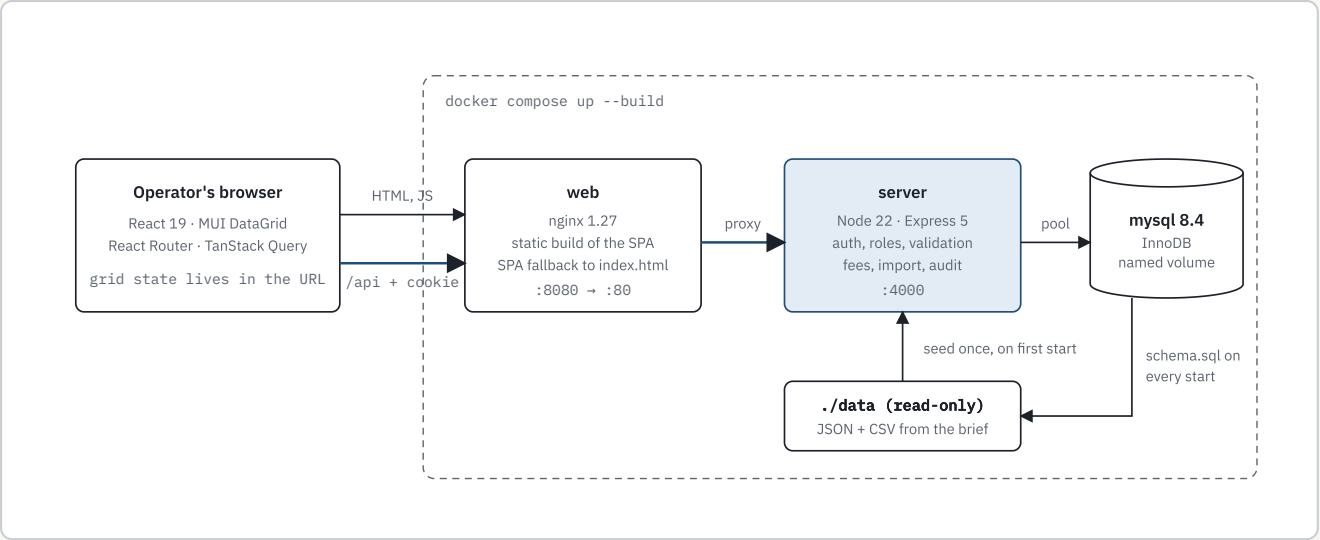
1. The problem

About 2,000 clients hold 3,030 portfolios. Fifty thousand transactions from January to September 2026 arrive from the custodian bank and need checking. Every month each client is billed from a fixed fee schedule. Six staff use the app all day, with three levels of authority:

Role	Users	May do
Viewer	Julia Wagner, Samir Haddad	See everything, change nothing
Operator	Tobias Brandt, Lea Fischer, Marco Rossi	Edit clients, approve or cancel transactions, generate and issue invoices, import files
Admin	Anna Keller	All of the above, plus mark invoices paid or void them

The brief's hard requirements, beyond the features themselves: links must reopen the exact grid view; grids must stay fast on the full data; two operators editing one client must not overwrite each other silently; bulk actions must explain what happened to each row; invoices must reconcile to the cent and be unique per client and month; imports must survive the same file being uploaded twice.

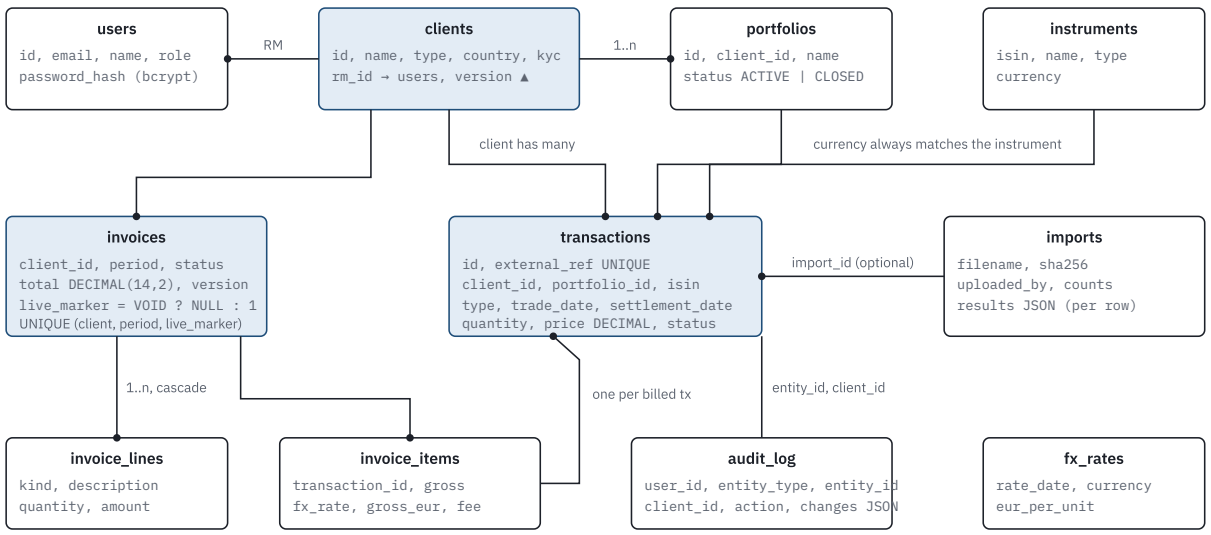
2. System architecture



One command starts all three services. The browser only ever talks to nginx; nginx serves the built React app and forwards `/api` to the Node server, which owns the schema and loads the brief's data into MySQL the first time it runs.

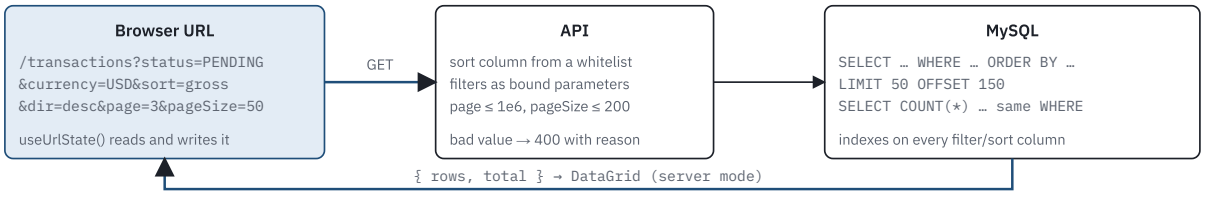
Why this shape: a single origin removes CORS and lets the session cookie be `SameSite=Strict`, which also covers CSRF. The server applies `schema.sql` idempotently on each start and seeds once, guarded by a marker row taken with `FOR UPDATE`, so even two server instances starting together load the data exactly once.

3. Data model



Money columns are `DECIMAL`, never floats. The invoice uniqueness rule is enforced in the database through a generated column: `live_marker` is `NULL` for void invoices, so the unique key only constrains the ones that count. Every table that grids filter or sort on carries an index for it.

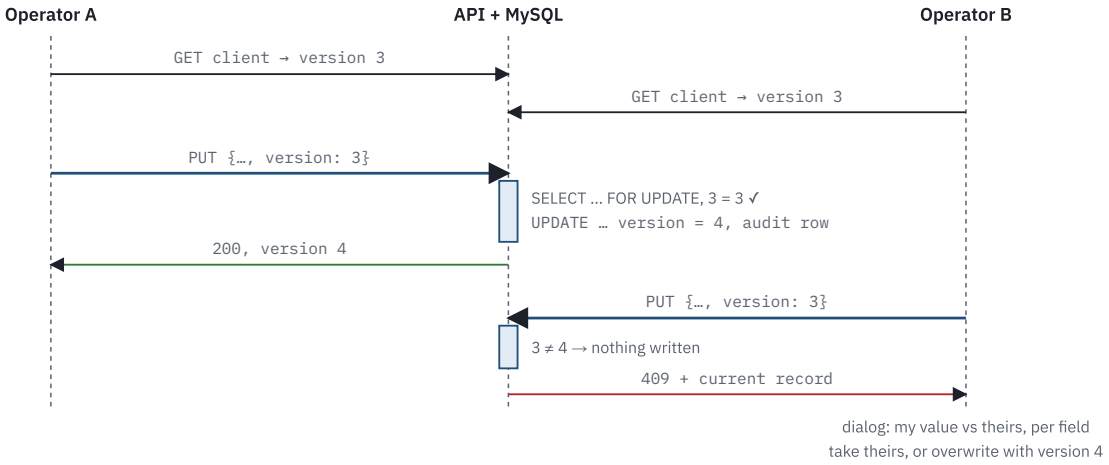
4. Grids: the URL is the state



Nothing about the view lives only in component state. Changing a filter rewrites the query string; opening a pasted link rebuilds the same request. The same filter builder also produces the CSV export, so the export is exactly what is on screen.

Each client tab is a nested route (`/clients/C-00001/transactions?status=SETTLED`), and a deep link opened while signed out returns to that exact location after login. Pagination is server-side; a page of 50 rows deep into the 50,000 transactions returns in well under 100 ms.

5. Two operators, one client: optimistic locking

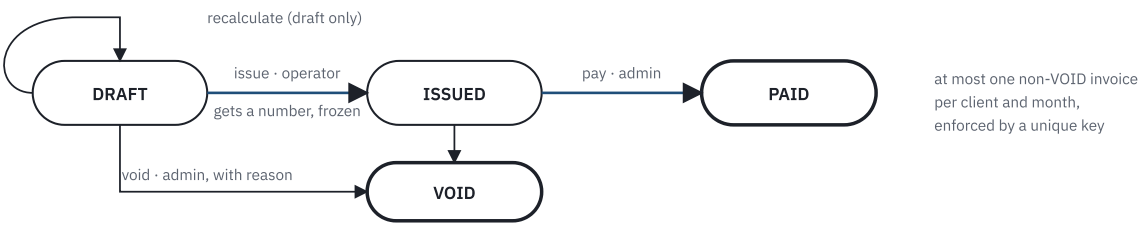


A version number travels with every edit. The update only applies when the row still has the version the editor started from; otherwise the API returns the current record and the UI shows a field-by-field comparison. Nothing is lost silently, and the loser decides.

6. Bulk approve and cancel

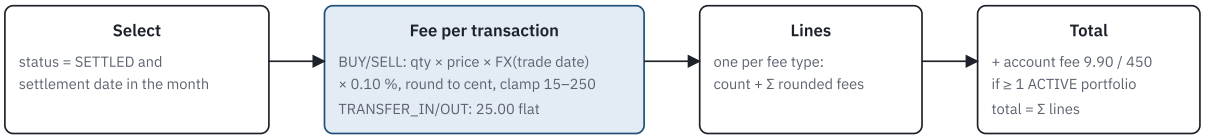
Operators tick rows and press Approve or Cancel. The server locks the selected rows (`SELECT … FOR UPDATE`), updates only those still `PENDING` , writes one audit entry per changed row, and returns an outcome for every requested id: `UPDATED` , `SKIPPED` (with its current status) or `NOT_FOUND` . The UI lists these in a table. Because the update is conditional on `status = 'PENDING'` under a row lock, two operators acting on the same transaction at once cannot both succeed; the second sees it skipped with the new status.

7. Invoicing



Each transition locks the invoice row, checks the allowed source state and the caller's permission, and is written to the audit log. Issuing re-computes the invoice from current data and refuses if the lines changed since the draft, so an issued invoice is correct at the moment it is frozen.

How the amount is computed



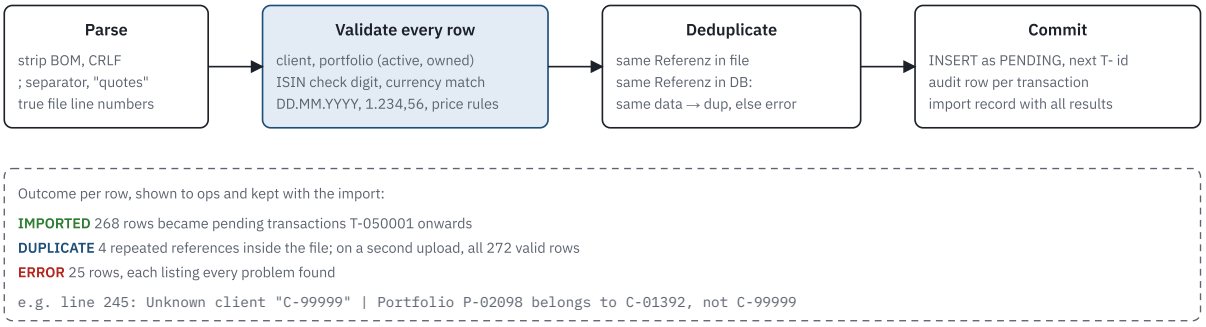
Worked example, March 2026, retail client C-00001:

```
3 buys 351.47 + 1 sell 25.26 + 1 transfer in 25.00 + 2 transfers out 50.00 + account fee 9.90 = 461.63 EUR
```

Every per-transaction figure (gross, FX rate, EUR gross, fee) is stored with the invoice, so each cent can be traced.

All arithmetic uses decimal.js and `DECIMAL` columns. Because each fee is rounded first and the lines and total are sums of already-rounded values, the invoice reconciles to the cent by construction rather than by a final rounding step.

8. Custodian import



Outcome per row, shown to ops and kept with the import:

IMPORTED 268 rows became pending transactions T-050001 onwards
DUPLICATE 4 repeated references inside the file; on a second upload, all 272 valid rows
ERROR 25 rows, each listing every problem found

```
e.g. line 245: Unknown client "C-99999" | Portfolio P-02098 belongs to C-01392, not C-99999
```

The whole run is one database transaction under a named lock, so two simultaneous uploads cannot both insert. Uploading the same file again produces zero new transactions and points to the earlier upload. Stray whitespace is trimmed; anything else that deviates from the specified format is reported rather than guessed.

9. Security and roles

Sessions

- Passwords stored as bcrypt hashes (cost 12)
- JWT in an `HttpOnly`, `SameSite=Strict` cookie, 10 h
- Unknown emails compared against a dummy hash, so timing reveals nothing
- 5 failures lock that address + email for 5 minutes

Authorisation

- Every `/api` route requires a session; the user is reloaded per request
- Each write route names a permission; roles map to permission lists on the server
- The UI reads the same list only to hide buttons; the server decides
- Sort columns are whitelisted; all values are bound parameters

Permission	Viewer	Operator	Admin
Read all screens, export CSV	yes	yes	yes
client:edit	—	yes	yes
transaction:status (approve, cancel)	—	yes	yes
invoice:generate, invoice:issue	—	yes	yes
import:run	—	yes	yes
invoice:pay, invoice:void	—	—	yes

10. Audit trail

Every client edit, transaction status change (including imports) and invoice transition writes an `audit_log` row on the same database connection as the change, so the two commit or roll back together. Each row records who, when, the entity, the action and a field-level `{ from, to }` diff. The client page's Audit log tab shows them newest first with links to the transaction or invoice involved.

11. How it was verified

- 16 unit tests on the two pure modules: fee maths (rounding, clamps, FX, reconciliation) and the import parser and validator.
- Every API route exercised as viewer, operator and admin with valid and malformed input.
- Four races run in parallel: concurrent client edits, approve vs cancel of one transaction, three simultaneous invoice generations, two uploads of the same file. Each produced exactly one winner and clean 409 responses.

- A SQL check across all generated invoices: total = Σ lines = Σ item fees + account fee, every fee within 15–250 EUR.
- A scripted browser walkthrough (Playwright on Chrome) against the Docker build from a wiped database.
- `docker compose down -v && docker compose up --build` from scratch: 50,000 transactions loaded in about ten seconds.

12. Known gaps and the path to production

Not done

- No automated API or end-to-end tests in the repo (verified by scripts, not CI)
- Audit log is per client, latest 1,000 entries, no global view
- No invoice PDF; portfolios and users are read-only
- Single JS bundle, no code splitting

Before real client data

- Secrets from a vault, TLS, security headers
- Revocable server-side sessions, MFA for admins
- Versioned migrations run at deploy time
- CI: lint, unit, integration (testcontainers), e2e
- Structured logs, metrics, readiness probe, backups

Source and setup instructions: the repository README. Brief: ASSIGNMENT.md. Data and rules: data/README.md.